GameManager

Aplicación web desarrollada con Laravel para la gestión completa de un catálogo de videojuegos, desarrolladoras, fichas técnicas y géneros.

Relaciones Eloquent

La aplicación implementa los tres tipos de relaciones principales de Eloquent:

1. Uno a uno (One to One)

- **Videojuego - Ficha Técnica**: Un videojuego posee una única ficha técnica con sus especificaciones técnicas (`plataforma`, `motor_grafico`, `edad_recomendada`, `espacio_gb`, `modo_juego`).
- Métodos utilizados:
  - `Videojuego::hasOne(FichaTecnica::class)`
  - `FichaTecnica::belongsTo(Videojuego::class)`

2. Uno a muchos (One to Many)

- **Desarrolladora - Videojuegos**: Una desarrolladora puede crear múltiples videojuegos, mientras que cada videojuego pertenece a una sola desarrolladora.
- Métodos utilizados:
  - `Desarrolladora::hasMany(Videojuego::class)`
  - `Videojuego::belongsTo(Desarrolladora::class)`

3. Muchos a muchos (Many to Many)

- **Videojuegos - Géneros**: Un videojuego puede estar clasificado bajo múltiples géneros y un género agrupa múltiples videojuegos. La relación se gestiona mediante la tabla intermedia `genero_videojuego`.
- Métodos utilizados:
  - `Videojuego::belongsToMany(Genero::class)->withTimestamps()`

- `Genero::belongsToMany(Videojuego::class)->withTimestamps()`
- Vinculación y actualización en controladores mediante `attach()` y `sync()`.

Factories y Seeders

El proyecto incluye fábricas para la generación automática de datos de prueba coherentes:

- `DesarrolladoraFactory`: Genera nombres de compañías, países y años de fundación.
- `VideojuegoFactory`: Genera títulos, descripciones, años de lanzamiento, precios y estado de publicación.
- `FichaTecnicaFactory`: Genera configuraciones técnicas con motores gráficos, plataformas y clasificaciones PEGI válidas.
- `GeneroFactory`: Permite la creación individual de géneros.
- `DatabaseSeeder`: Orquesta el poblado inicial creando 8 géneros predeterminados, 5 desarrolladoras y asignando entre 2 y 4 videojuegos por desarrolladora con sus respectivas fichas técnicas y entre 1 y 3 géneros aleatorios.

Eager Loading y Prevención del Problema N+1

Para garantizar la eficiencia en el acceso a la base de datos y evitar consultas redundantes:

- **Listado principal**: Se utiliza carga impaciente (*eager loading*) en `VideojuegoController::index`:

  ```php
  Videojuego::with(['desarrolladora', 'generos'])->latest()->paginate(10);
  ```

  Esto reduce las consultas de $1 + 2N$ a solo 3 consultas SQL optimizadas para toda la colección.

- **Detalle y edición**: Se emplea `load()` tras el enlace implícito de rutas (*Route Model Binding*):

```
$videojuego->load(['desarrolladora', 'fichaTecnica', 'generos']);
```

- **Contador de desarrolladoras**: En `DesarrolladoraController::index` se emplea `withCount('videojuegos')` para computar los títulos asociados sin cargar los modelos completos en memoria.

Comandos de Instalación y Ejecución

Para desplegar y ejecutar el entorno localmente:

1. Instalar dependencias de Composer:

```
composer install
```

2. Generar la clave de la aplicación:

```
php artisan key:generate
```

3. Ejecutar las migraciones junto con el seeder de datos iniciales:

```
php artisan migrate:fresh --seed
```

4. Iniciar el servidor local: